Prompting Large Language Models with Hugging Face

The three foundational chat roles: System, User, and Assistant. Crafting precise, reliable prompts that unlock the full potential of LLMs

Why Prompting Matters

Prompting serves as the primary interface to control LLM behavior, functioning as a bridge between human intent and machine output. It determines the tone, structure, and quality of responses, making it essential for building reliable, repeatable AI systems.

Before diving into advanced tools, agents, or fine-tuning approaches, mastering prompting fundamentals establishes a critical foundation. Think of prompting as **programming with language** — a new paradigm where precise linguistic instructions replace traditional code syntax.

The Three Chat Roles

Chat-based language models utilize three distinct roles to structure conversations.

System

Establishes the model's identity, operational guidelines, and behavioral constraints. Sets tone, defines expertise level, and implements guardrails for output safety and relevance.

User

Contains the actual request, question, or task from the human operator. This is where you specify what you want the model to do, including any contextual information needed for completion.

Assistant

Represents the model's generated response. We don't manually write this role's content — the LLM produces it based on the system instructions and user request.

Key Insight: Roles are architectural patterns for organizing prompts, not hardcoded features of transformer models. They provide structure and context that guides generation.

Prompt Structure Pattern

Anatomy of a Complete Prompt

A well-structured prompt follows a clear three-part pattern. The system message defines behavioral parameters, the user message contains the specific request, and the assistant slot awaits the model's response.

01

Define Behavior

System role establishes identity and constraints

02

Ask Clearly

User role specifies the exact task

03

Expect Structure

Assistant generates the structured response

Example Prompt Structure

```
system: "You are a precise and helpful teaching assistant."
user: "Explain attention in transformers in two bullet points."
assistant: (model fills this)
```

Hugging Face Chat API

Hugging Face provides a clean, intuitive API for interacting with chat-based language models. The `InferenceClient` class handles authentication, model selection, and message formatting, allowing you to focus on prompt engineering rather than infrastructure.

```
from huggingface_hub import InferenceClient

client = InferenceClient("mistralai/Mistral-7B-Instruct-v0.2")

messages = [
                {"role": "system", "content": "You are a clear CS tutor."},
                {"role": "user", "content": "Explain transformers in one sentence."}
]

response = client.chat.completions.create(
    model="mistralai/Mistral-7B-Instruct-v0.2",
    messages=messages,
    temperature=0.0,
    max_tokens=120,
)

print(response.choices[0].message["content"])
```

Key Parameters

`messages`: List of role-content dictionaries following system → user pattern

`temperature`: Controls randomness (0.0 = deterministic, 1.0 = creative)

`max_tokens`: Limits response length for cost and latency control

Prompting Best Practices

Effective prompting requires systematic application of proven techniques. These practices transform vague requests into precise, actionable instructions that yield consistent, high-quality outputs.

1 Define a Clear Role

Start with "You are..." to establish model identity and expertise level. Example: "You are a senior software architect specializing in distributed systems."

2 Specify Explicit Constraints

Set boundaries for length, format, and style. Define output structure (bullets, JSON, numbered steps) and prohibit unwanted behaviors like external information retrieval or speculation.

3 Iterate Continuously

Treat prompting as a debugging process. Test variations, measure outputs, and refine instructions based on observed behavior. Small changes can produce significant improvements.

Bad → Better: Prompt Evolution

Examining the progression from poor to excellent prompts reveals the impact of systematic refinement.
Each iteration adds clarity, structure, and constraints that guide the model toward desired outputs.

1

Bad

Explain attention./

✗ No context, no role, no constraints

2

Better

System: You are a CS tutor.**User:** Explain attention in transformers.

⚠ Better, but lacks output constraints

3

Even Better

System: You are a CS tutor for 4th year college CS students.**User:** Explain attention in transformers in 3 short bullets, <20 words each.

Clear role, explicit constraints, structured output

What Changed?

1. Added system role defining expertise
2. Specified output format (3 bullets)
3. Set length constraint (<20 words each)

Hands-On Exercise

Build a Prompt Step-by-Step (5 minutes)

Start Simple

Begin with a basic user prompt asking about positional encoding

Add System Role

Define model identity as a precise CS educator

Add Constraints

Specify format (2 bullets), length (concise), and style (no equations)

Primary Task

Ask the model to explain "positional encoding" clearly, in exactly 2 bullets, with no mathematical equations. Focus on conceptual understanding appropriate for senior engineers.

Stretch Goal: Experiment with temperature values of 0.0 (deterministic) and 0.7 (balanced) to observe how randomness affects explanation style and word choice.